

স্বয়ংক্রিয় এআই-চালিত অ্যান্ড্রয়েড অ্যাপ্লিকেশন ইঞ্জিন: মোবাইল-কেন্দ্রিক সফটওয়্যার ডেভেলপমেন্ট আর্কিটেকচার

বর্তমান সফটওয়্যার ইঞ্জিনিয়ারিংয়ের প্রেক্ষাপটে কৃত্রিম বুদ্ধিমত্তা বা এআই (AI) শুধুমাত্র কোড লেখার সহায়ক হিসেবে সীমাবদ্ধ নেই; এটি এখন সম্পূর্ণ স্বয়ংক্রিয় ডেভেলপমেন্ট ইকোসিস্টেম বা "এজেন্টিক সফটওয়্যার ইঞ্জিনিয়ারিং" (Agentic Software Engineering) মডেলে রূপান্তরিত হয়েছে। একটি প্রথাগত কম্পিউটার বা ল্যাপটপের উপর নির্ভরশীলতা দূর করে, শুধুমাত্র একটি অ্যান্ড্রয়েড স্মার্টফোন (যেমন- POCO X3) ব্যবহার করে সম্পূর্ণ স্বয়ংক্রিয় 'প্রাইভেট ফ্যাক্টরি' বা অ্যাপ্লিকেশন ইঞ্জিন তৈরি করার ধারণাটি প্রযুক্তিগত দিক থেকে অত্যন্ত উদ্ভাবনামূলক এবং যুগান্তকারী। এই গবেষণাপত্রের মূল লক্ষ্য হলো ক্লাউড বিলিংয়ের জটিলতা সম্পূর্ণ এড়িয়ে, গুগল রেন (Gemini API / Jules), গিটহাব অ্যাকশনস (GitHub Actions), ফায়ারবেস অ্যাপ ডিস্ট্রিবিউশন (Firebase App Distribution), এবং টারমাক্স (Termux)-এর সমন্বয়ে একটি এন্ড-টু-এন্ড (End-to-End) স্বয়ংক্রিয় এপিকে (APK) জেনারেশন আর্কিটেকচার বিশ্লেষণ ও প্রস্তুত করা। ব্যবহারকারীর মূল চাহিদা অনুযায়ী, একটি ওয়েব অ্যাপ্লিকেশনের জিপ (ZIP) ফাইল বা অ্যাপ্লিকেশনের ক্লিপট/আইডিয়া ইনপুট দিলে, এআই এজেন্ট স্বয়ংক্রিয়ভাবে কোড জেনারেট করবে, কম্পাইল করবে এবং চূড়ান্ত এপিকে (APK) ফাইল হিসেবে টারমাক্সের মাধ্যমে সরাসরি মোবাইল ডিভাইসে ইনস্টল করবে। এই প্রক্রিয়ায় ফায়ারবেস স্টুডিও (Firebase Studio) বনাম লোকাল টারমাক্স (Termux) বিস্তারিত উপযোগিতা, Model Context Protocol (MCP) সার্ভারগুলোর কার্যকারিতা এবং ম্যানুয়াল বনাম স্বয়ংক্রিয় কনফিগারেশনের একটি বিশদ, ধাপে ধাপে রেডি-টু-ইউজ (Ready-to-use) গাইডলাইন নিচে উপস্থাপন করা হলো।

আর্কিটেকচারাল মূল্যায়ন: ক্লাউড-ভিত্তিক বনাম লোকাল টারমাক্স (Termux) বিল্ড

একটি স্বয়ংক্রিয় অ্যাপ্লিকেশন ইঞ্জিন নির্মাণের ক্ষেত্রে প্রথম এবং সবচেয়ে গুরুত্বপূর্ণ সিদ্ধান্ত হলো বিল্ড বা কম্পাইলেশন প্রক্রিয়ার রানটাইম পরিবেশ (Runtime Environment) নির্ধারণ করা। এ বিষয়ে দুটি প্রধান বিকল্প বিদ্যমান: ক্লাউড-ভিত্তিক পরিবেশ (যেমন ফায়ারবেস স্টুডিও বা এআই স্টুডিও) এবং সম্পূর্ণ লোকাল টারমাক্স (Termux) পরিবেশ। ব্যবহারকারীর নির্দেশনা অনুযায়ী, নিচে উভয় পদ্ধতির জন্যই প্রযুক্তিগত গাইডলাইন এবং তুলনামূলক উপযোগিতা বিশ্লেষণ করা হলো।

১. ক্লাউড-ভিত্তিক সমাধান: ফায়ারবেস স্টুডিও (Firebase Studio) এবং এআই স্টুডিও

ফায়ারবেস স্টুডিও (Firebase Studio) গুগলের একটি ব্রাউজার-ভিত্তিক, এআই-চালিত ডেভেলপমেন্ট পরিবেশ যা প্রজেক্ট আইডিএক্স (Project IDX) এর উত্তরসূরি হিসেবে আত্মপ্রকাশ করেছিল। এটি জেমিনি (Gemini) মডেলের সাহায্যে ন্যাচারাল ল্যাঙ্গুয়েজ প্রম্পট থেকে সরাসরি অ্যাপ্লিকেশন প্রোটোটাইপ তৈরি করতে সক্ষম। ফায়ারবেস স্টুডিওর বর্তমান অবস্থা ও ভবিষ্যৎ: গবেষণা থেকে প্রাপ্ত তথ্য অনুযায়ী, গুগল আনুষ্ঠানিকভাবে ঘোষণা করেছে যে ফায়ারবেস স্টুডিও ২২ মার্চ, ২০২৭ তারিখে সম্পূর্ণরূপে বন্ধ (Sunset) হতে যাচ্ছে। এর ধারাবাহিকতায় ২২ জুন, ২০২৬ থেকে এই প্ল্যাটফর্ম নতুন ওয়ার্কস্পেস তৈরি করা বা নতুন ব্যবহারকারী নিবন্ধন সম্পূর্ণভাবে বন্ধ করে দেওয়া হয়েছে। গুগল তাদের এআই ডেভেলপার টুলগুলোকে একত্রিত করে গুগল এআই স্টুডিও (Google AI Studio) এবং গুগল অ্যান্টিগ্র্যাভিটি (Google Antigravity)-তে স্থানান্তর করেছে। যেহেতু ফায়ারবেস স্টুডিও একটি অবলুপ্ত পথে থাকা প্ল্যাটফর্ম, তাই এর উপর ভিত্তি করে একটি দীর্ঘমেয়াদী ব্যক্তিগত অ্যাপ্লিকেশন ইঞ্জিন তৈরি করা কোনোভাবেই বাস্তবসম্মত বা টেকসই কৌশল হতে পারে না।

২. অ্যান্ড্রয়েড ফোনে লোকাল টারমাক্স (Termux) দ্বারা বিল্ড প্রক্রিয়া

বিকল্প হিসেবে POCO X3 এর মতো ডিভাইসে টারমাক্সের (Termux) মাধ্যমে সম্পূর্ণ লোকাল পরিবেশে অ্যান্ড্রয়েড অ্যাপ্লিকেশন বিন্ধ করার কৌশলটি বিবেচনা করা যেতে পারে। টারমাক্স মূলত অ্যান্ড্রয়েডের জন্য একটি শক্তিশালী লিনাক্স এমুলেটর, যেখানে গ্রেডল (Gradle) এবং অ্যান্ড্রয়েড এসডিকে (Android SDK) লোকালি ইনস্টল করে এপিকে বিন্ধ করা সম্ভব।

লোকাল টারমাক্স বিন্ধের জন্য রেডি-টু-ইউজ নির্দেশিকা: টারমাক্সে অ্যান্ড্রয়েড বিন্ধ এনভায়রনমেন্ট সেটআপ করার জন্য নিম্নলিখিত ধাপগুলো অনুসরণ করতে হয় :

১. প্রাথমিক প্যাকেজ ইনস্টলেশন: টারমাক্স চালু করে বেস প্যাকেজ এবং জাভা ডেভেলপমেন্ট কিট (JDK) ইনস্টল করতে হবে। গ্রেডল ৮.১০.২ বা আধুনিক সংস্করণের জন্য জাভা ২১ বা ১৭ প্রয়োজন।

```
pkg update && pkg upgrade
pkg install wget unzip git
pkg install openjdk-17 aapt aapt2
```

২. অ্যান্ড্রয়েড এসডিকে (**Android SDK**) সেটআপ: গুগলের কমান্ড-লাইন টুলস ডাউনলোড করে এসডিকে ডিরেক্টরি প্রস্তুত করতে হবে।

```
mkdir -p $HOME/android/sdk/cmdline-tools
wget
https://dl.google.com/android/repository/commandlinetools-linux-9477386_latest.zip
unzip commandlinetools-linux-*.zip -d $HOME/android/sdk/cmdline-tools
mv $HOME/android/sdk/cmdline-tools/cmdline-tools
$HOME/android/sdk/cmdline-tools/latest
```

৩. এনভায়রনমেন্ট ভেরিয়েবল (**Environment Variables**) কনফিগারেশন: টারমাক্সের ~/.bashrc ফাইলে পাথগুলো যুক্ত করতে হবে যাতে টারমাক্স এসডিকে টুলস চিনতে পারে।

```
echo 'export ANDROID_HOME=$HOME/android/sdk' >> ~/.bashrc
echo 'export PATH=$PATH:$ANDROID_HOME/cmdline-tools/latest/bin' >>
~/.bashrc
echo 'export PATH=$PATH:$ANDROID_HOME/platform-tools' >> ~/.bashrc
source ~/.bashrc
```

৪. এসডিকে ম্যানেজার দিয়ে প্ল্যাটফর্ম ইনস্টল:

```
yes | sdkmanager "platform-tools" "platforms;android-34"
"build-tools;34.0.0"
```

৫. **AAPT2** বাইনারি ইস্যু সমাধান: টারমাক্সে গ্রেডল চালানোর সময় সবচেয়ে বড় বাধা হলো aapt2 (Android Asset Packaging Tool) ইস্যু। গ্রেডল যে ডিফল্ট লিনাক্স বাইনারি নামায় তা টারমাক্সের ARM64 আর্কিটেকচারে (POCO X3) চলে না। এর জন্য টারমাক্সের নিজস্ব aapt2 দিয়ে গ্রেডলের ক্যাশে থাকা বাইনারিকে প্রতিস্থাপন করতে হয় :

```
fi[span_17](start_span)[span_17](end_span)nd ~/.gradle -name
'aapt2-*-linux.jar' -type f | xargs -I{} jar -u -f {} -C
/data/data/com.termux/files/usr/bin aapt2
```

৬. এপিকে বিন্ধ করা: এরপর সোর্স কোড ডিরেক্টরিতে গিয়ে গ্রেডল কমান্ড চালাতে হয়:

```
./gradlew assembleDebug
```

লোকাল টারমাক্স পদ্ধতির সীমাবদ্ধতা ও বিশ্লেষণ: যদিও উপরোক্ত পদ্ধতিতে লোকালি এপিকে বিন্ধ করা সম্ভব, তবে একটি স্বয়ংক্রিয় 'অ্যাপ্লিকেশন ইঞ্জিন' এর জন্য এটি মোটেও আদর্শ নয়। POCO X3 এর স্ল্যাপড্রাগন প্রসেসর এবং র‍্যাম যখন গ্রেডল বিন্ধ প্রসেস চালায়, তখন তা ডিভাইসের সিপিইউ এবং ব্যাটারির উপর চরম চাপ প্রয়োগ করে। থার্মাল থ্রটলিংয়ের কারণে বিন্ধ হতে দীর্ঘ সময় লাগে। এছাড়া, এআই এজেন্টকে টারমাক্সের লোকাল স্টোরেজে রান করানো এবং

তার সাথে এসডিকে সিস্টেম করানো অত্যন্ত জটিল একটি কাজ।

৩. এক্সপার্ট সুপারিশ: ক্লাউড-লোকাল হাইব্রিড আর্কিটেকচার

ফায়ারবেস স্টুডিওর আসন্ন সমাপ্তি এবং লোকাল টারমাক্স বিল্ডের হার্ডওয়্যার সীমাবদ্ধতা বিবেচনা করে, সবচেয়ে নিখুঁত, দ্রুত এবং নির্ভরযোগ্য সমাধান হলো একটি "ক্লাউড-লোকাল হাইব্রিড প্রাইভেট ফ্যাক্টরি" তৈরি করা। এই মডেলে, POCO X3 এর টারমাক্স শুধুমাত্র একটি "কন্ট্রোলার" বা "কমান্ড সেন্টার" হিসেবে কাজ করবে। গুগলের ব্রেন (Gemini / Jules API) কোড লেখার চিন্তন প্রক্রিয়া পরিচালনা করবে। গিটহাবের সার্ভার (GitHub Actions) কোডগুলোকে কম্পাইল করে এপিকে (APK) তৈরি করবে। এবং ফায়ারবেস অ্যাপ ডিস্ট্রিবিউশন সেই এপিকে ফাইলটিকে টারমাক্সে ডেলিভারি করবে। যেহেতু গিটহাব অ্যাকশনস পাবলিক রিপোজিটরির জন্য সম্পূর্ণ ফ্রি এবং প্রাইভেট রিপোজিটরির জন্য প্রতি মাসে ২০০০ মিনিট ফ্রি কম্পিউটেশনাল টাইম প্রদান করে, তাই গুগলের ক্লাউড প্ল্যাটফর্ম (GCP) ক্রেডিট কার্ড যুক্ত করার বা বিলিং অ্যাকাউন্টের কোনো ঝামেলাই এখানে থাকবে না। এটিই হলো কোনো বিলিং ছাড়াই এপিকে জেনারেট করার পিওর ফ্লো (Pure Flow)।

জোলস (Jules) প্ল্যাটফর্ম এবং Model Context Protocol (MCP) মেকানিজম

জোলস (Jules) হলো গুগল ল্যাবস কর্তৃক তৈরি একটি স্বায়ত্তশাসিত (Autonomous), এজেন্টিক কোডিং অ্যাসিস্ট্যান্ট যা সরাসরি গিটহাব রিপোজিটরির সাথে যুক্ত হয়ে কাজ করে। এটি প্রচলিত কোডিং এআইদের মতো নয়, বরং এটি একটি সম্পূর্ণ ভার্সুয়াল মেশিন (VM) স্পিন আপ করে, রিপোজিটরি ক্লোন করে, কোড লেখে, ডিপেন্ডেন্সি আপডেট করে, টেস্ট রান করে এবং গিটহাবে কমিট করে।

স্ক্রিনশট অনুযায়ী MCP লিস্ট বিশ্লেষণ এবং ইন্টিগ্রেশন কৌশল

ব্যবহারকারীর উল্লেখিত Jules.google MCP.jpg স্ক্রিনশটে থাকা MCP (Model Context Protocol) সার্ভারগুলো জোলসের কার্যক্ষমতাকে বহুগুণ বাড়িয়ে দেয়। জোলস jules.google.com প্ল্যাটফর্মে প্রিমিয়াম অ্যাকাউন্টের আওতাধীন হওয়ায়, এই এমসিপিগুলো ব্যবহার করে একটি পূর্ণাঙ্গ অ্যাপ্লিকেশন ইঞ্জিন তৈরি করার বিশাল সুযোগ রয়েছে। স্ক্রিনশটে থাকা এমসিপিগুলোর স্কোপ এবং আমাদের পরিকল্পনার সাথে তাদের ম্যাচিং নিচে বিশ্লেষণ করা হলো:

MCP সার্ভারের নাম	অফিশিয়াল স্ট্যাটাস	আমাদের অ্যাপ ইঞ্জিনের সাথে স্কোপ ম্যাচিং এবং প্রয়োজনীয়তা
Context7	Connected	এটি সম্ভবত একটি কাস্টম বা লোকাল কনটেক্সট প্রোভাইডার। এটি প্রজেক্টের রিকোয়ারমেন্টস এবং ইউজারের প্রিফারেন্স জোলসকে বোঝাতে সাহায্য করবে। এটি সক্রিয় রাখা আবশ্যিক।
Render MCP	Connected	Render API এর মাধ্যমে ডিপ্লয়মেন্ট ম্যানেজ করে। যদি অ্যাপ্লিকেশনটির কোনো ব্যাকএন্ড নোড.জেএস (Node.js) বা পাইথন সার্ভার থাকে, তবে জোলস এই এমসিপি দিয়ে সরাসরি রেন্ডারে তা ডিপ্লয় করতে পারবে। নেটিভ এপিকে জেনারেশনের জন্য এটি ঐচ্ছিক (Optional)।
Linear	Official	সফটওয়্যার প্রজেক্ট ম্যানেজমেন্ট এবং ইস্যু ট্র্যাকিং টুল। আমাদের প্রাইভেট ফ্যাক্টরিতে এআই এজেন্টকে যদি টিকিট

MCP সার্ভারের নাম	অফিশিয়াল স্ট্যাটাস	আমাদের অ্যাপ ইঞ্জিনের সাথে স্কেপ ম্যাচিং এবং প্রয়োজনীয়তা
		বা টাস্ক ভিত্তিক কাজ দিতে হয়, তবে লিনিয়ার এমসিপি জোলসকে স্বয়ংক্রিয়ভাবে কাজগুলোর স্ট্যাটাস আপডেট করতে সাহায্য করবে। ব্যক্তিগত প্রজেক্টের জন্য এটি ঐচ্ছিক।
Neon	Official	সার্ভারলেস পোস্টগ্রেস (Serverless Postgres) ডাটাবেস। যদি তৈরিকৃত অ্যান্ড্রয়েড অ্যাপ্লিকেশনে ক্লাউড ডাটাবেস প্রয়োজন হয়, তবে জোলস এই এমসিপি ব্যবহার করে স্বয়ংক্রিয়ভাবে ডাটাবেস স্কিমা তৈরি এবং ম্যানেজ করতে পারবে।
Stitch	Official	অত্যন্ত গুরুত্বপূর্ণ: এটি গুগল ল্যাবসের এআই-পাওয়ার্ড ইউআই (UI) ডিজাইন টুল। টেক্সট প্রম্পট থেকে সরাসরি ইউআই স্ক্রিন জেনারেট করা এবং DESIGN.md ফাইল তৈরি করার জন্য এটি এই আর্কিটেকচারের অন্যতম ভিত্তি। এটি অবশ্যই সক্রিয় করতে হবে।
Supabase	Official	ওপেন সোর্স ফায়ারবেস অল্টারনেটিভ (সার্ভারলেস পোস্টগ্রেস)। অ্যাপ্লিকেশনের অথেনটিকেশন, ডাটাবেস এবং স্টোরেজের জন্য জোলস স্বয়ংক্রিয়ভাবে ডাটাবেস টেবিল এবং এপিআই (API) তৈরি করতে এটি ব্যবহার করবে। এটি সক্রিয় করা সুপারিশকৃত।
Tinybird	Official	রিয়েল-টাইম অ্যানালিটিক্স প্ল্যাটফর্ম। আমাদের অ্যাপ ইঞ্জিনের কোর ফিচারের জন্য এটি প্রয়োজনীয় নয়। এটি স্কিপ করা যেতে পারে।
v0	Official	ভার্সেল (Vercel) এর এআই-পাওয়ার্ড ইউআই জেনারেটর। স্টিচ (Stitch) এমসিপির একটি শক্তিশালী বিকল্প হিসেবে এটি ব্যবহার করা যেতে পারে। আধুনিক ওয়েব-টু-অ্যান্ড্রয়েড অ্যাপের রিয়েক্ট (React) ভিত্তিক ইউআই তৈরির জন্য এটি সক্রিয় করা অত্যন্ত লাভজনক।

প্রিমিয়াম অ্যাকাউন্টের কৌশলগত সুবিধা: জোলস প্রিমিয়াম অ্যাকাউন্টে কনকারেন্সি (একসাথে একাধিক কাজ করার ক্ষমতা) এবং টাস্ক লিমিট অনেক বেশি থাকে। বর্তমানে সাধারণ ব্যবহারকারীদের জন্য প্রতিদিন ৬০টি টাস্ক এবং ৫টি কনকারেন্ট সেশন সীমাবদ্ধ করা হয়েছে। প্রিমিয়াম অ্যাকাউন্টের সাহায্যে আমরা [google-labs-code/jules-skills](https://github.com/google-labs/code-jules-skills) এর [automate-github-issues](https://github.com/google-labs/code-jules-skills/issues) স্ক্রিপ্ট ব্যবহার করে গিটহাবের ইস্যু ট্র্যাকারকে একটি মাল্টি-এজেন্ট প্রম্পট ইনবক্স হিসেবে ব্যবহার করতে পারব, যেখানে একাধিক এজেন্ট সমান্তরালভাবে (Parallel) কাজ করবে।

필수 MCP: savethepolarbears/jules-mcp-server

টারমাফ্র বা গিটহাব অ্যাকশনস থেকে জোলসকে নিয়ন্ত্রণ করার জন্য savethepolarbears/jules-mcp-server নামক এমসিপি সার্ভারটি আর্কিটেকচারের জন্য একটি মাস্টার-কী (Master Key) হিসেবে কাজ করে। এটি জোলস v1alpha API-এর সম্পূর্ণ কভারেজ প্রদান করে, যার মাধ্যমে /sessions (নতুন কাজ তৈরি), /sessions/{id} (কাজের স্ট্যাটাস দেখা), এবং /sessions/{id}:approvePlan (কাজের প্ল্যান অ্যাপ্রুভ করা) এন্ডপয়েন্টগুলো সরাসরি টারমাফ্র থেকে হিট করা যায়।

কনফিগারেশন ম্যাট্রিক্স: জোলস কী করতে পারে এবং ম্যানুয়াল প্রস্তুতি

জোলস এমসিপি ইন্টিগ্রেশন সম্পন্ন হওয়ার পর, এআই এজেন্ট স্বয়ংক্রিয়ভাবে একটি অ্যাপ্লিকেশন ডেভলপমেন্টের অধিকাংশ কাজ করে ফেলতে পারে। তবে একটি 'প্রাইভেট ফ্যাক্টরি' দাঁড় করানোর জন্য ইঞ্জিনিয়ারকে (ব্যবহারকারীকে) প্রথমবার কিছু নির্দিষ্ট ইনফ্রাস্ট্রাকচার ম্যানুয়ালি কনফিগার করে দিতে হবে। নিচে এর একটি পূর্ণাঙ্গ বিভাজন দেওয়া হলো:

ম্যানুয়ালি যে কনফিগারেশনগুলো রেডি করে দিতে হবে (One-time Setup)

১. **Jules API Key** জেনারেশন: jules.google.com/settings#api পোর্টালে গিয়ে প্রিমিয়াম অ্যাকাউন্টের জন্য একটি এপিআই কী (API Key) তৈরি করতে হবে। এই কী-টি গিটহাব রিপোজিটরির Settings -> Secrets and variables -> Actions এ গিয়ে JULES_API_KEY নামে সংরক্ষণ করতে হবে। ২. গিটহাব অ্যাকশনস ওয়ার্কফ্লো (YAML): এআই কীভাবে কোড বিন্ড করবে এবং ফায়ারবেসে পাঠাবে, তার ইনস্ট্রাকশন সেট বা YAML ফাইলটি ম্যানুয়ালি গিটহাবে রাখতে হবে। ৩. ফায়ারবেস সার্ভিস অ্যাকাউন্ট (Firebase Service Account): ফায়ারবেস কনসোল থেকে প্রজেক্ট তৈরি করে অ্যাপ ডিস্ট্রিবিউশন (App Distribution) চালু করতে হবে। এরপর GCP কনসোল থেকে সার্ভিস অ্যাকাউন্টের JSON ফাইলটি ডাউনলোড করে গিটহাব সিক্রেটসে FIREBASE_CREDENTIALS_JSON হিসেবে রাখতে হবে এবং অ্যাপ আইডিটি FIREBASE_APP_ID হিসেবে রাখতে হবে। ৪. টারমাফ্র এবং শিজুকু এনভায়রনমেন্ট: মোবাইল ডিভাইসে টারমাফ্র এবং শিজুকু (Shizuku) ইনস্টল করে এডিবি (ADB) পেয়ারিং সম্পন্ন করতে হবে, যাতে টারমাফ্র ব্যাকগ্রাউন্ডে এপিকে ইনস্টল করার প্রিভিলেজ পায়। ৫. অ্যাপ্রুয়েড প্রজেক্ট স্কেলিটন (Skeleton): গিটহাব রিপোজিটরিতে একটি বেসিক ওয়েব-টু-অ্যাপ্রুয়েড (যেমন Capacitor, Cordova বা Flutter) এর ক্ল্যাক প্রজেক্ট স্ট্রাকচার রাখতে হবে, যার উপর ভিত্তি করে জোলস কাজ শুরু করবে।

জোলস (Jules) যে কনফিগারেশন এবং কাজগুলো স্বয়ংক্রিয়ভাবে করবে

১. ডিজাইন সিস্টেম তৈরি: প্রম্পট পাওয়ার পর জোলস প্রথমে Stitch বা v0 এমসিপি ব্যবহার করে সম্পূর্ণ ইউজার ইন্টারফেসের কালার, টাইপোগ্রাফি এবং রুলস সম্বলিত একটি .stitch/DESIGN.md ফাইল তৈরি করবে। ২. কোড লজিক এবং আর্কিটেকচার: ইনপুটকৃত ক্লিয়ার বা জিপ ফাইলের ভিত্তিতে জোলস তার ক্লাউড ভার্সুয়াল মেশিনে কোড লিখবে, পেজ রাউটিং ঠিক করবে এবং প্রয়োজনীয় ডিপেন্ডেন্সি (package.json বা build.gradle) আপডেট করবে। ৩. কোড কমিট এবং গিটহাব পুশ: টাস্ক শেষ হলে জোলস স্বয়ংক্রিয়ভাবে গিটহাবের মেইন (main) ব্রাঞ্চে কোডগুলো পুশ করে দেবে এবং একটি পুল রিকোয়েস্ট (PR) ওপেন করবে। ৪. এপিকে বিন্ড এবং ডেলিভারি ট্রিগার: জোলস কোড পুশ করার সাথে সাথেই গিটহাব অ্যাকশনস স্বয়ংক্রিয়ভাবে ট্রিগার হবে, যা গ্রেডল ব্যবহার করে এপিকে বিন্ড করবে এবং ফায়ারবেসে আপলোড করবে।

রেডি-টু-ইউজ পূর্ণাঙ্গ গাইড: ক্লিক-বাই-ক্লিক এক্সিকিউশন ফ্লো

নিচে সম্পূর্ণ আর্কিটেকচারটির এন্ড-টু-এন্ড (End-to-End) বাস্তবায়নের রেডি-টু-ইউজ গাইডলাইন প্রদান করা হলো, যা সরাসরি এক্সিকিউট করা যাবে।

ধাপ ১: গিটহাব অ্যাকশনস ইঞ্জিন সেটআপ (The Build & Delivery Pipeline)

আপনার গিটহাব রিপোজিটরিতে `.github/workflows/ai_factory_engine.yml` নামক একটি ফাইল তৈরি করুন এবং নিচের কোডটি হুবহু পেস্ট করুন। এটিই আপনার ক্লাউড অ্যাপ্লিকেশন ইঞ্জিনের মূল হার্টবিট।

```
name: AI Private Factory Engine

on:
  workflow_dispatch:
    inputs:
      ai_prompt:
        description: 'The blueprint or raw idea for the application'
        required: true
        default: 'Build a task management mobile app UI'

jobs:
  jules_cognition_phase:
    name: Phase 1 - AI Code Generation (Jules)
    runs-on: ubuntu-latest
    permissions:
      contents: write
      pull-requests: write
    steps:
      - name: Checkout Code Repository
        uses: actions/checkout@v3

      - name: Invoke Jules Coding Agent
        uses: google-labs-code/jules-invoke@v1
        with:
          prompt: "You are an expert mobile developer. Read DESIGN.md
rules. Implement the following idea into the web-to-apk project
structure. Idea: ${ github.event.inputs.ai_prompt }"
          jules_api_key: ${ secrets.JULES_API_KEY }
          starting_branch: 'main'
          include_last_commit: false

  compilation_and_delivery_phase:
    name: Phase 2 - Build APK & Firebase Distribution
    needs: jules_cognition_phase
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Generated Code
        uses: actions/checkout@v3
        with:
          ref: 'main'

      - name: Set up Java JDK 17
        uses: actions/setup-java@v3
```

```

with:
  distribution: 'zulu'
  java-version: '17'

- name: Make Gradle Wrapper Executable
  run: chmod +x./gradlew

- name: Compile and Build Android APK
  run: ./gradlew assembleRelease --no-daemon

- name: Distribute APK via Firebase
  uses: wzieba/Firebase-Distribution-Github-Action@v1
  with:
    appId: ${{ secrets.FIREBASE_APP_ID }}
    serviceCredentialsFileContent: ${{
secrets.FIREBASE_CREDENTIALS_JSON }}
    groups: testers
    file: app/build/outputs/apk/release/app-release.apk

```

লজিক্যাল ব্যাখ্যা: এই ওয়াইএএমএল (YAML) ফাইলটি দুটি ধাপে কাজ করে। প্রথম ধাপে (jules_cognition_phase), workflow_dispatch এর মাধ্যমে আসা প্রম্পটটি google-labs-code/jules-invoke@v1 অ্যাকশনের মাধ্যমে জোলসের কাছে পাঠানো হয়। জোলস কোড লিখে গিটহাবে সেভ করে। দ্বিতীয় ধাপে (compilation_and_delivery_phase), গিটহাবের উবুন্টু রানার লেটেস্ট কোডটি ডাউনলোড করে গ্রেডল (Gradle) এর মাধ্যমে অ্যান্ড্রয়েড এপিকে কম্পাইল করে। এরপর wzieba/Firebase-Distribution-Github-Action@v1 অ্যাকশনটি স্বয়ংক্রিয়ভাবে সেই এপিকে ফায়ারবেস অ্যাপ ডিস্ট্রিবিউশনে পাঠিয়ে দেয়।

ধাপ ২: টারমাক্স (Termux) এবং শিজুকু (Shizuku) সেটআপ (The Receiving End)

POCO X3 ডিভাইসে ফায়ারবেস থেকে এপিকে রিসিভ করে সাইলেন্টলি (কোনো ইনস্টল পপ-আপ ছাড়া) ইনস্টল করার জন্য এই মেকানিজমটি অপরিহার্য। এটি অ্যান্ড্রয়েডের সিকিউরিটি লেয়ার বাইপাস করে টারমাক্সকে সিস্টেম লেভেল প্রিভিলেজ দেয়।

১. গুগল প্লে স্টোর বা এফ-ড্রয়েড (F-Droid) থেকে **Termux** এবং **Shizuku** অ্যাপ ইনস্টল করুন। ২. POCO X3 এর Settings -> Developer Options এ গিয়ে Wireless Debugging চালু করুন। ৩. টারমাক্স ওপেন করে বেসিক টুলস ইনস্টল করুন:

```

pkg update && pkg upgrade
pkg install curl jq android-tools termux-setup-storage
termux-setup-storage

```

৪. শিজুকু পেয়ারিং: টারমাক্সে অডিবি (ADB) কানেক্ট করার জন্য ওয়ারারলেস ডিবাগিং স্ক্রিন থেকে পেয়ারিং কোড এবং পোর্ট নম্বর নিয়ে টারমাক্সে কমান্ড দিন:

```
adb pair localhost:PORT "PAIRING_CODE"
```

৫. পেয়ারিং সফল হলে শিজুকু অ্যাপটি খুলুন এবং Start via Wireless Debugging এ ক্লিক করুন। ৬. শিজুকু অ্যাপের Use Shizuku in terminal apps অপশনে গিয়ে টারমাক্স ডিরেক্টরিতে ফাইল এক্সপোর্ট করুন। এরপর টারমাক্সে টাইপ করুন:

```
shizuku
```

এটি টারমাক্সকে থিজুর সাথে যুক্ত করবে এবং rish নামক একটি শেল স্ক্রিপ্ট টারমাক্সে ইনস্টল হবে। এই rish কমান্ডটি ব্যবহার করেই আমরা পরবর্তীতে pm install কমান্ডটি চালাব।

ধাপ ৩: টারমাক্স প্রাইভেট ফ্যাক্টরি অটোমেশন স্ক্রিপ্ট (The Master Controller)

টারমাক্সে একটি ব্যাশ (Bash) স্ক্রিপ্ট তৈরি করতে হবে, যা সম্পূর্ণ ইঞ্জিনটিকে একটি মাত্র ইনপুট প্রম্পট থেকে নিয়ন্ত্রণ করবে। টারমাক্সে nano app-engine.sh টাইপ করে নিচের স্ক্রিপ্টটি সেভ করুন।

```
#!/bin/bash

# =====
# AI App Engine - Mobile Private Factory
# =====

# Configuration (আপনার নিজস্ব ডাটা দিয়ে রিপ্লেস করুন)
GITHUB_PAT="ghp_your_personal_access_token_here"
GITHUB_OWNER="your_github_username"
GITHUB_REPO="your_repository_name"
WORKFLOW_ID="ai_factory_engine.yml"
FIREBASE_DOWNLOAD_URL="your_static_firebase_latest_release_url"
# ফায়ারবেস থেকে একটি স্ট্যাটিক ডাইরেক্ট ডাউনলোড লিংক এখানে ব্যবহার করতে হবে

echo "=====
echo "      Welcome to Personal AI App Engine      "
echo "=====
echo ""
echo "Please describe the App Blueprint or idea:"
read AI_APP_PROMPT

echo ""
echo "[1/4] Sending instructions to Jules Cloud Brain via GitHub
Actions..."

# curl ব্যবহার করে GitHub REST API কল করে workflow_dispatch ট্রিগার করা
curl -s -X POST \
  -H "Accept: application/vnd.github+json" \
  -H "Authorization: Bearer $GITHUB_PAT" \
  -H "X-GitHub-Api-Version: 2022-11-28" \

https://api.github.com/repos/$GITHUB_OWNER/$GITHUB_REPO/actions/workfl
ows/$WORKFLOW_ID/dispatches \
  -d
{"ref\":\"main\", \"inputs\":{ \"ai_prompt\": \"$AI_APP_PROMPT\" } }"

echo " Jules has started processing the prompt."
echo "Waiting for Jules to write code, GitHub to compile, and Firebase
to deliver..."
echo "This usually takes 8-15 minutes. Polling for delivery..."
```



```
# একটি সিম্পল স্লিপ মেকানিজম (প্রোডাকশনে ফায়ারবেস এপিআই পোলিং করা যেতে পারে)
sleep 600

APK_SAVE_PATH="/sdcard/Download/ai_generated_factory_app.apk"

echo "[2/4] Downloading compiled APK from Firebase App
Distribution..."
curl -L -o "$APK_SAVE_PATH" "$FIREBASE_DOWNLOAD_URL"

echo "[3/4] Silently installing the APK directly into POCO X3..."
# Shizuku-এর rish ব্যবহার করে অ্যান্ড্রয়েড প্যাকেজ ম্যানেজার (pm) এর মাধ্যমে ইনস্টলেশন
sh rish -c "pm install -r $APK_SAVE_PATH"

echo "[4/4] Installation Complete! The app is ready to launch on your
device."
```

লজিক্যাল ব্যাখ্যা: এই স্ক্রিপ্টটিই হলো সম্পূর্ণ আর্কিটেকচারের ব্রেন-ইন্টারফেস। এটি ব্যবহারকারীর কাছ থেকে অ্যাপের আইডিয়া ইনপুট নেয়। এরপর curl কমান্ড ব্যবহার করে গিটহাবের dispatches এন্ডপয়েন্টে একটি POST রিকোয়েস্ট পাঠায়, যা গিটহাব অ্যাকশনস ওয়ার্কফ্লোকে ট্রিগার করে। ওয়ার্কফ্লো শেষ হওয়ার পর এটি ফায়ারবেস থেকে এপিকেটি টারমিনাল ডাউনলোড করে। সর্বশেষ জাদুকরী ধাপটি হলো sh rish -c "pm install -r \$APK_SAVE_PATH", যা শিজুকুর এডিবি পাওয়ার ব্যবহার করে অ্যান্ড্রয়েড সিস্টেমে কোনো কনফার্মেশন পপ-আপ ছাড়াই (Silent Install) অ্যাপটি ইনস্টল করে দেয়।

ধাপ ৪: এআই প্রম্পট ইঞ্জিনিয়ারিং এবং DESIGN.md স্পেসিফিকেশন

জোলসকে দিয়ে সঠিকভাবে কাজ করানোর জন্য একটি স্ট্রাকচার্ড (Structured) প্রম্পট এবং ডিজাইন স্পেসিফিকেশন অত্যন্ত জরুরি। সাধারণ প্রম্পট দিলে এআই এলোমেলো ইউজার ইন্টারফেস তৈরি করতে পারে, যাকে 'এআই স্লপ' (AI slop) বলা হয়। গুগল ল্যাবস এর সমাধান হিসেবে DESIGN.md স্পেসিফিকেশন ওপেন-সোর্স করেছে। গিটহাব রিপোজিটরির মূলে .stitch/DESIGN.md নামক একটি ফাইল রাখতে হবে। এই ফাইলে প্রজেক্টের কালার কোড, ফন্ট সাইজ, স্পেসিং ইত্যাদি মার্কডাউন (Markdown) ফরম্যাটে লেখা থাকবে। রেডি-টু-ইউজ প্রম্পট টেমপ্লেট (যা টারমিনাল স্ক্রিপ্টে ইনপুট দিতে হবে):

"Activate the **Stitch MCP (stitch-design)** and **Supabase MCP**. First, analyze the requested app blueprint: [আপনার আইডিয়া এখানে দিন]. Generate a robust .stitch/DESIGN.md file encapsulating a modern, minimalist design system with explicitly defined hex codes, typography layers, and responsive CSS grids. Once the design system is synthesized, use the create_coding_task tool to write the frontend code matching these rules exactly. Set up a Supabase database schema for user authentication. Do not use generic AI styles; adhere strictly to the DESIGN.md directives."

এই প্রম্পটটি জোলসকে বাধ্য করবে প্রথমে এমসিপি সার্ভারগুলোর সাথে কানেক্ট করতে, ডিজাইনের রুলবুক তৈরি করতে এবং তারপর সেই রুলবুক মেনে কোড লিখতে।

যৌক্তিক সমাধান হিসেবে আর্কিটেকচারটির মূল্যায়ন ও সীমাবদ্ধতা

উপরোক্ত আর্কিটেকচারটি যৌক্তিকভাবে সবচেয়ে নিখুঁত এবং কার্যকর সমাধান। ফায়ারবেস স্টুডিওর বিলুপ্তি এবং লোকাল টারমিনাল কম্পাইলেশনের চরম হার্ডওয়্যার সীমাবদ্ধতাকে পাশ কাটিয়ে এটি একটি ক্লাউড-নেটিভ কিন্তু লোকালি নিয়ন্ত্রিত ফ্যাক্টরি তৈরি করে। এর ফলে POCO X3 এর ব্যাটারি, স্টোরেজ বা প্রসেসরের ওপর কোনো চাপ পড়ে না,

অথচ পুরো সিস্টেমটি মোবাইল থেকেই নিয়ন্ত্রিত হয়।

সীমাবদ্ধতা ও সতর্কতা: ১. জোলস এপিআই রোট লিমিট: জোলস বর্তমানে প্রতিদিন সর্বোচ্চ ৬০টি টাস্ক সম্পাদনের সুযোগ দেয়। তবে ব্যক্তিগত অ্যাপ ইঞ্জিনের জন্য এই সীমাটি যথেষ্ট। ২. ফায়ারবেস সার্ভিস অ্যাকাউন্ট সিকিউরিটি: গিটহাব সিক্রেটসে রাখা FIREBASE_CREDENTIALS_JSON এর জেসন ডেটা অত্যন্ত সংবেদনশীল। এটি পাবলিক রিপোজিটরিতে লিক হলে সিকিউরিটি ব্রীচ হতে পারে। ৩. শিজুকু পারসিস্টেন্স: ডিভাইস রিবুট (Reboot) বা রিস্টার্ট করা হলে শিজুকুর ওয়্যারলেস ডিবাগিং পারমিশন চলে যায়। তাই ফোন রিস্টার্ট করলে টারমাক্সে একবার adb pair প্রক্রিয়াটি পুনরায় করতে হবে। ৪. **aapt2** এবং নেটিভ বিল্ড এরর: গিটহাব অ্যাকশনস এ উবুন্টু রানার ব্যবহার করায় টারমাক্সের কুখ্যাত aapt2 আর্কিটেকচারাল এরর (Architecture Error) সম্পূর্ণভাবে এড়ানো সম্ভব হয়েছে।

উপসংহার

বিশ্লেষণ থেকে এটি দ্ব্যর্থহীনভাবে প্রমাণিত যে, প্রচলিত কোনো ল্যাপটপ বা ক্লাউড বিলিং অ্যাকাউন্টের কার্ড অ্যাড করার ঝামেলা ছাড়াই, শুধুমাত্র POCO X3 এবং টারমাক্স ব্যবহার করে একটি এআই নিয়ন্ত্রিত 'প্রাইভেট ফ্যাক্টরি' বা অ্যাপ্লিকেশন ইঞ্জিন তৈরি করা শতভাগ সম্ভব। এর জন্য লোকাল টারমাক্সে কোড বিল্ড করার প্রাচীন এবং ত্রুটিপূর্ণ পদ্ধতি (যাতে aapt2 এরর এবং ওভারহিটিং হয়) পরিহার করতে হবে।

পরিবর্তে, জোলস এপিআই (Jules API) এবং স্টিচ এমসিপি (Stitch MCP) এর চিন্তাশক্তি, গিটহাব অ্যাকশনসের অসীম এবং ফ্রি ক্লাউড বিল্ড পাওয়ার, ফায়ারবেস অ্যাপ ডিস্ট্রিবিউশনের ডেলিভারি পাইপলাইন, এবং টারমাক্স-শিজুকুর (Shizuku) সাইলেন্ট ইন্সটলেশন মেকানিজম—এই চারটি স্তম্ভকে একসাথে জোড়া লাগিয়ে যে হাইব্রিড আর্কিটেকচারটি প্রস্তাব করা হয়েছে, তা একজন ইঞ্জিনিয়ারের জন্য চূড়ান্ত এবং স্বয়ংক্রিয় মোবাইল-কেন্দ্রিক সফটওয়্যার ডেভলপমেন্ট ইকোসিস্টেম হিসেবে কাজ করবে। এটি শুধুমাত্র এপিকে জেনারেটই করবে না, বরং DESIGN.md এবং এমসিপি মেকানিজমের কল্যাণে পেশাদার মানের ইউজার ইন্টারফেস এবং আর্কিটেকচারাল নিশ্চয়তাও প্রদান করবে।

যেসব কাজের উদ্ধৃতি যোগ করা হয়েছে

1. Firebase Studio, <https://firebase.studio/> 2. Firebase Studio Is Shutting Down\ What You Need to Do Before March 2027 - Reddit, https://www.reddit.com/r/Firebase/comments/1rysgyk/firebase_studio_is_shutting_down_what_you_need_to/ 3. Firebase Studio sunset and project migration - Google, <https://firebase.google.com/docs/studio/migrating-project> 4. Google announces it is shutting down Firebase Studio, less than a year after its launch; read the message to users | - The Times of India, <https://timesofindia.indiatimes.com/technology/tech-news/google-announces-it-is-shutting-down-firebase-studio-less-than-a-year-after-its-launch-read-the-message-to-users/articleshow/129694408.cms> 5. illegalvoid/termux-build-apps: build APKs with Termux, without needing Proot - GitHub, <https://github.com/illegalvoid/termux-build-apps> 6. What are we missing to build android apps using termux? Is it possible? - Reddit, https://www.reddit.com/r/termux/comments/1od59bk/what_are_we_missing_to_build_android_apps_using/ 7. Build android app on termux - GitHub Gist, <https://gist.github.com/tmatz/817bf03433e059bf89c63dc33f286ccb> 8. Building Android applications in Termux using Gradle - Grokipedia, https://grokipedia.com/page/Building_Android_applications_in_Termux_using_Gradle 9. Built an Android Studio plugin that handles the entire Firebase App Distribution pipeline — build, sign, upload, distribute in one click - Reddit, https://www.reddit.com/r/Firebase/comments/1ueckds/built_an_android_studio_plugin_that_handles_the/ 10. Complete Guide: Setting Up CI/CD for Android Apps with GitHub Actions, Firebase & Play Store | by Satyam Jha | Medium, <https://medium.com/@satyamkrjha85/complete-guide-setting-up-ci-cd-for-android-apps-with-gith>

ub-actions-firebase-play-store-5b942208473a 11. Build with Jules, your asynchronous coding agent - Google Blog, <https://blog.google/innovation-and-ai/models-and-research/google-labs/jules/> 12. Changelog | Jules, <https://jules.google/docs/changelog/> 13. jules-skills/automate-github-issues by google-labs-code - High Risk | Security Scan, <https://mondoo.com/ai-agent-security/skills/github/google-labs-code/jules-skills/automate-github-issues/7a76d3bfcf42> 14. automate-github-issues | Agent Skills Library - Awesome MCP Servers, <https://mcpservers.org/agent-skills/google-labs-code/automate-github-issues> 15. Production-ready Model Context Protocol (MCP) server for Google Jules API. Enables autonomous coding tasks and scheduling from AI assistants. Independent open-source project, not affiliated with Google. · GitHub, <https://github.com/savethepolarbears/jules-mcp-server> 16. Jules MCP 伺服器 - LobeHub, <https://lobehub.com/zh-TW/mcp/savethepolarbears-jules-mcp-server> 17. Level Up Your Dev Game: The Jules API is Here! - Google Developers Blog, <https://developers.googleblog.com/en/level-up-your-dev-game-the-jules-api-is-here/> 18. google-labs-code/jules-action: Add a powerful cloud coding agent to your GitHub workflows, <https://github.com/google-labs-code/jules-action> 19. BeksOmega/jules-action: Start sessions on jules.google.com from GitHub workflows, <https://github.com/BeksOmega/jules-action> 20. A quick guide to GitHub Actions CI/CD for Android — Firebase Distribution & Play Store : r/androiddev - Reddit, https://www.reddit.com/r/androiddev/comments/1r7l109/a_quick_guide_to_github_actions_cicd_for_android/ 21. Firebase App Distribution · Actions · GitHub Marketplace, <https://github.com/marketplace/actions/firebase-app-distribution> 22. A guide to implement CD in Android projects using GitHub Actions and Firebase App Distribution | by Arthur Henz | CodeX | Medium, <https://medium.com/codex/a-guide-to-implement-cd-in-android-projects-using-github-actions-and-firebase-app-distribution-c491a672874a> 23. Automatically activate Shizuku in Android 14+ (non-root) #462 - GitHub, <https://github.com/RikkaApps/Shizuku/discussions/462> 24. Shizuku gives your Termux ADB (Shell) privileges to remove bloatware, list running processes, open listening ports, view stored Wi-Fi passwords, inspect logcat of other apps, enable/disable specific Android app components etc. Details - Reddit, https://www.reddit.com/r/termux/comments/1lzkjku/shizuku_gives_your_termux_adb_shell_privileges_to/ 25. Google Stitch + Claude Code via MCP: From Design to Code Without Copy-Paste, <https://felixschmidt.software/en/blog/google-stitch-mcp-claude-code> 26. GitHub - google-labs-code/design.md: A format specification for describing a visual identity to coding agents. DESIGN.md gives agents a persistent, structured understanding of a design system., <https://github.com/google-labs-code/design.md> 27. Google makes DESIGN.md open source on its way to become a industry standard - Medium, <https://medium.com/design-bootcamp/google-makes-design-md-open-source-on-its-way-to-become-a-industry-standard-16119f2368dd> 28. Setting Up Flutter CI/CD with GitHub Actions and Firebase App Distribution Securely, <https://dev.to/heyroziq/setting-up-flutter-cicd-with-github-actions-and-firebase-app-distribution-securely-346a> 29. Jules Invoke · Actions · GitHub Marketplace, <https://github.com/marketplace/actions/jules-invoke> 30. This action uploads artifacts (.apk or .ipa) to Firebase App Distribution. - GitHub, <https://github.com/wzieba/Firebase-Distribution-Github-Action> 31. Soft de-google guide (no rooted device nor custom ROM required) : r/degoogle - Reddit, https://www.reddit.com/r/degoogle/comments/1p1fozp/soft_degoogle_guide_no_rooted_device_

nor_custom/ 32. Question: are there limitations of using Shizuku? · Issue #15 - GitHub, <https://github.com/RikkaApps/Shizuku-API/issues/15> 33. You don't need a PC for ADB anymore—here's how to do it in Termux - How-To Geek, <https://www.howtogeek.com/you-dont-need-a-pc-for-adb-anymoreheres-how-to-do-it-in-termux/> 34. How can I run ADB commands using Shizuku? #404 - GitHub, <https://github.com/RikkaApps/Shizuku/discussions/404> 35. Taking (almost) full control of your unrooted Android - for Customization, and Privacy (Long, extensive) : r/privacytoolsIO - Reddit, https://www.reddit.com/r/privacytoolsIO/comments/fit0tr/taking_almost_full_control_of_your_unrooted/ 36. Workflow Dispatch · Actions · GitHub Marketplace, <https://github.com/marketplace/actions/workflow-dispatch> 37. REST API endpoints for workflows - GitHub Docs, <https://docs.github.com/en/rest/actions/workflows?apiVersion=2026-03-10> 38. How to trigger Github Action's workflow dispatch event through curl with string input in json format - Stack Overflow, <https://stackoverflow.com/questions/75286485/how-to-trigger-github-actions-workflow-dispatch-event-through-curl-with-string> 39. [til/github-actions/trigger-a-workflow-via-an-api-call.md](https://github.com/til/github-actions/blob/master/trigger-a-workflow-via-an-api-call.md) at master, <https://github.com/jbranchaud/til/blob/master/github-actions/trigger-a-workflow-via-an-api-call.md> 40. App Manager - Muntashir Al-Islam, <https://muntashirakon.github.io/AppManager/pt-rBR/> 41. pass-with-high-score/universal-installer: A modern Android app for installing and managing APK packages with split APK support, silent install via Shizuku, and VirusTotal malware scanning. - GitHub, <https://github.com/pass-with-high-score/universal-installer> 42. android-developer-verification-discourse.md - GitHub Gist, <https://gist.github.com/agnostic-apollo/b8d8daa24cbdd216687a6bef53d417a6> 43. Using Claude Code + Google Stitch to handle UI design for client projects, it's wonderful, https://www.reddit.com/r/ClaudeCode/comments/1s8hpd/using_claude_code_google_stitch_to_handle_ui/ 44. Stitch's DESIGN.md format is now open-source so you can use it across platforms. - Google Blog, <https://blog.google/innovation-and-ai/models-and-research/google-labs/stitch-design-md/>